UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Examen Parcial II

Programación

Israel Roberto Soto García

CARNET: 9941-25-226

GUATEMALA, 25 ABRIL 2026.

Codigo

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <limits> // Para limpiar el buffer de entrada

using namespace std;

// Estructura para representar una venta
struct Venta {
    int idVenta;
    string producto;
    double precio;
    int cantidad;
    double total;
};

// Declaración de funciones
void mostrarMenu();
void registrarVenta(vector<Venta>& ventas, int& nextId);
void guardarVentas(const vector<Venta>& ventas);
void cargarVentas(vector<Venta>& ventas, int& nextId);
void consultarVentas();
void generarReporte();
```

```
const string NOMBRE_ARCHIVO = "ventas.dat";

int main() {
    vector<Venta> listaVentas;
    int nextId = 1; // Para asignar IDs únicos a las ventas

    // Cargar ventas existentes al iniciar el programa
    cargarVentas(listaVentas, nextId);

    int opcion;
    do {
        mostrarMenu();
        cout << "Seleccione una opción: ";
        cin >> opcion;

        // Limpiar el buffer de entrada después de leer un número
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        switch (opcion) {
            case 1:
                registrarVenta(listaVentas, nextId);
                break;
            case 2:
                consultarVentas();
```

```
            break;
        case 3:
            generarReporte();
            break;
        case 4:
            guardarVentas(listaVentas);
            cout << "Saliendo del programa. ¡Hasta luego!" << endl;
            break;
        default:
            cout << "Opción no válida. Por favor, intente de nuevo." << endl;
    }
    cout << endl; // Espacio entre operaciones
  } while (opcion != 4);

  return 0;
}

// Procedimiento para mostrar el menú principal
void mostrarMenu() {
  cout <<
"==========================================================
=============================================" << endl;
  cout << "   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute
nuevamente el programa" << endl;
  cout <<
"==========================================================
=============================================" << endl;
```

```
    cout << "   Sistema de Control de Ventas" << endl;
    cout << "==============================" << endl;
    cout << "1. Registrar Venta" << endl;
    cout << "2. Consultar Ventas" << endl;
    cout << "3. Generar Reporte" << endl;
    cout << "4. Guardar y Salir" << endl;
    cout << "------------------------------" << endl;
}

// Función para registrar una nueva venta
void registrarVenta(vector<Venta>& ventas, int& nextId) {
    Venta nuevaVenta;
    nuevaVenta.idVenta = nextId;

    cout << "\n--- Registro de Nueva Venta ---" << endl;
    cout << "ID de Venta: " << nuevaVenta.idVenta << endl;

    cout << "Ingrese el nombre del producto: ";
    getline(cin, nuevaVenta.producto);

    cout << "Ingrese el precio unitario: ";
    cin >> nuevaVenta.precio;
    // Limpiar buffer después de leer el precio
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
    cout << "Ingrese la cantidad vendida: ";
    cin >> nuevaVenta.cantidad;
    // Limpiar buffer después de leer la cantidad
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    nuevaVenta.total = nuevaVenta.precio * nuevaVenta.cantidad;
    cout << "Total de la venta: $" << nuevaVenta.total << endl;

    ventas.push_back(nuevaVenta);
    nextId++; // Incrementar el ID para la próxima venta

    cout << "Venta registrada exitosamente." << endl;
}

// Procedimiento para guardar todas las ventas en un archivo
void guardarVentas(const vector<Venta>& ventas) {
    ofstream archivoSalida(NOMBRE_ARCHIVO, ios::binary | ios::trunc); //
Abrir en modo binario y truncar si existe

    if (!archivoSalida.is_open()) {
        cerr << "Error: No se pudo abrir el archivo para guardar ventas." << endl;
        return;
    }

    // Escribir la cantidad de ventas al principio del archivo
    size_t numVentas = ventas.size();
```

```
    archivoSalida.write(reinterpret_cast<const char*>(&numVentas),
sizeof(numVentas));


    for (const auto& venta : ventas) {
        archivoSalida.write(reinterpret_cast<const char*>(&venta.idVenta),
sizeof(venta.idVenta));
        archivoSalida.write(reinterpret_cast<const char*>(&venta.precio),
sizeof(venta.precio));
        archivoSalida.write(reinterpret_cast<const char*>(&venta.cantidad),
sizeof(venta.cantidad));
        archivoSalida.write(reinterpret_cast<const char*>(&venta.total),
sizeof(venta.total));


        // Escribir la longitud del string del producto
        size_t longitudProducto = venta.producto.length();
        archivoSalida.write(reinterpret_cast<const char*>(&longitudProducto),
sizeof(longitudProducto));
        // Escribir los caracteres del string del producto
        archivoSalida.write(venta.producto.c_str(), longitudProducto);
    }


    archivoSalida.close();
    cout << "Ventas guardadas en '" << NOMBRE_ARCHIVO << "'." << endl;
}


// Procedimiento para cargar ventas desde un archivo
void cargarVentas(vector<Venta>& ventas, int& nextId) {
```

```
    ifstream archivoEntrada(NOMBRE_ARCHIVO, ios::binary);

    if (!archivoEntrada.is_open()) {
        // Si el archivo no existe, no hay problema, simplemente empezamos sin
ventas
        cout << "Archivo de ventas no encontrado. Se iniciará con un registro
vacío." << endl;
        return;
    }

    size_t numVentas;
    archivoEntrada.read(reinterpret_cast<char*>(&numVentas),
sizeof(numVentas));

    for (size_t i = 0; i < numVentas; ++i) {
        Venta ventaCargada;
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.idVenta),
sizeof(ventaCargada.idVenta));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.precio),
sizeof(ventaCargada.precio));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.cantidad),
sizeof(ventaCargada.cantidad));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.total),
sizeof(ventaCargada.total));

        size_t longitudProducto;
        archivoEntrada.read(reinterpret_cast<char*>(&longitudProducto),
sizeof(longitudProducto));
```

```
        ventaCargada.producto.resize(longitudProducto);
        archivoEntrada.read(&ventaCargada.producto[0], longitudProducto);

        ventas.push_back(ventaCargada);
        // Actualizar nextId para que sea mayor que el último ID cargado
        if (ventaCargada.idVenta >= nextId) {
            nextId = ventaCargada.idVenta + 1;
        }
    }

    archivoEntrada.close();
    cout << "Ventas cargadas desde '" << NOMBRE_ARCHIVO << "'." <<
endl;
}

// Función para consultar todas las ventas registradas
void consultarVentas() {
    cout << "\n--- Listado de Todas las Ventas ---" << endl;
    // Aquí necesitaríamos cargar las ventas desde el archivo de nuevo o
tenerlas en memoria
    // Para este ejemplo, asumimos que las ventas están en memoria si el
programa no ha sido cerrado.
    // Si se implementa la persistencia completa, aquí se leerían del archivo.

    // Si se desea mostrar ventas que están en el vector 'listaVentas' de main:
    // Se tendría que pasar el vector a esta función o tener una forma de acceder
a él.
```

```
    // Por simplicidad en este ejemplo, si el programa se reinicia, la consulta
mostraría vacío
    // hasta que se registren nuevas ventas.

    // Si quisieramos leer directamente del archivo para consultar:
    ifstream archivoEntrada(NOMBRE_ARCHIVO, ios::binary);
    if (!archivoEntrada.is_open()) {
        cout << "No hay ventas registradas o el archivo no se puede abrir." <<
endl;
        return;
    }

    size_t numVentas;
    archivoEntrada.read(reinterpret_cast<char*>(&numVentas),
sizeof(numVentas));

    if (numVentas == 0) {
        cout << "No hay ventas registradas." << endl;
        archivoEntrada.close();
        return;
    }

    cout << "ID | Producto       | Precio Unit. | Cantidad | Total" << endl;
    cout << "-----------------------------------------------------" << endl;
    for (size_t i = 0; i < numVentas; ++i) {
        Venta ventaCargada;
```

```
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.idVenta),
sizeof(ventaCargada.idVenta));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.precio),
sizeof(ventaCargada.precio));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.cantidad),
sizeof(ventaCargada.cantidad));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.total),
sizeof(ventaCargada.total));


        size_t longitudProducto;
        archivoEntrada.read(reinterpret_cast<char*>(&longitudProducto),
sizeof(longitudProducto));
        ventaCargada.producto.resize(longitudProducto);
        archivoEntrada.read(&ventaCargada.producto[0], longitudProducto);


        cout << ventaCargada.idVenta << "  | " << ventaCargada.producto << " |
$" << ventaCargada.precio << " | " << ventaCargada.cantidad << "      | $" <<
ventaCargada.total << endl;
    }
    archivoEntrada.close();
}

// Función para generar un reporte de ventas
void generarReporte() {
    cout << "\n--- Reporte General de Ventas ---" << endl;

    ifstream archivoEntrada(NOMBRE_ARCHIVO, ios::binary);
    if (!archivoEntrada.is_open()) {
```

```
        cout << "No hay ventas registradas para generar un reporte." << endl;
        return;
    }

    size_t numVentas;
    archivoEntrada.read(reinterpret_cast<char*>(&numVentas),
sizeof(numVentas));

    if (numVentas == 0) {
        cout << "No hay ventas registradas para generar un reporte." << endl;
        archivoEntrada.close();
        return;
    }

    double totalVentasGlobal = 0.0;
    // Opcional: Podríamos contar la cantidad de productos vendidos, o el
promedio de precio.

    cout << "Resumen de Ventas:" << endl;
    cout << "-------------------" << endl;

    for (size_t i = 0; i < numVentas; ++i) {
        Venta ventaCargada;
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.idVenta),
sizeof(ventaCargada.idVenta));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.precio),
sizeof(ventaCargada.precio));
```

```
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.cantidad),
sizeof(ventaCargada.cantidad));
        archivoEntrada.read(reinterpret_cast<char*>(&ventaCargada.total),
sizeof(ventaCargada.total));


        size_t longitudProducto;
        archivoEntrada.read(reinterpret_cast<char*>(&longitudProducto),
sizeof(longitudProducto));
        ventaCargada.producto.resize(longitudProducto);
        archivoEntrada.read(&ventaCargada.producto[0], longitudProducto);


        totalVentasGlobal += ventaCargada.total;
    }


    cout << "Número total de ventas registradas: " << numVentas << endl;
    cout << "Ingreso total por ventas: $" << totalVentasGlobal << endl;
    // Podríamos añadir más estadísticas aquí si fuera necesario.
    archivoEntrada.close();
}
```

Prueba

```
PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output> cd 'c:\Users\Israel\OneDrive\Deskto
PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output> & .\'PruevaII.exe'
Ventas cargadas desde 'ventas.dat'.
==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 4
Ventas guardadas en 'ventas.dat'.
Saliendo del programa. ┬íHasta luego!

PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output> cd 'c:\Users\Israel\OneDrive\Deskto
PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output> & .\'PruevaII.exe'
Ventas cargadas desde 'ventas.dat'.
==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 3

--- Reporte General de Ventas ---
No hay ventas registradas para generar un reporte.

==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 1
```

```
Seleccione una opci├│n: 1

--- Registro de Nueva Venta ---
ID de Venta: 1
Ingrese el nombre del producto: glovos
Ingrese el precio unitario: 0.50
Ingrese la cantidad vendida: 60
Total de la venta: $30
Venta registrada exitosamente.

==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 1

--- Registro de Nueva Venta ---
ID de Venta: 2
Ingrese el nombre del producto: cocacola
Ingrese el precio unitario: 6
Ingrese la cantidad vendida: 60
Total de la venta: $360
Venta registrada exitosamente.

==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 1

--- Registro de Nueva Venta ---
ID de Venta: 3
Ingrese el nombre del producto: clicles tridet
Ingrese el precio unitario: 0.5
Ingrese la cantidad vendida: 30
Total de la venta: $15
Venta registrada exitosamente.
```

```
==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 4
Ventas guardadas en 'ventas.dat'.
Saliendo del programa. ┬íHasta luego!

PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output> cd 'c:\Users\Israel\OneDrive\Desktop
PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output> & .\'PruevaII.exe'
Ventas cargadas desde 'ventas.dat'.
==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 2

--- Listado de Todas las Ventas ---
ID | Producto       | Precio Unit. | Cantidad | Total
-----------------------------------------------------
1  | glovos | $0.5 | 60       | $30
2  | cocacola | $6 | 60       | $360
3  | clicles tridet | $0.5 | 30       | $15

==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 3
```

```
==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 3

--- Reporte General de Ventas ---
Resumen de Ventas:
-------------------
N├╢mero total de ventas registradas: 3
Ingreso total por ventas: $405

==========================================================================================
   Para las opciones 2 y 3 primero guardar con el numero 4 y ejecute nuevamente el programa
==========================================================================================
   Sistema de Control de Ventas
==============================
1. Registrar Venta
2. Consultar Ventas
3. Generar Reporte
4. Guardar y Salir
------------------------------
Seleccione una opci├│n: 4
Ventas guardadas en 'ventas.dat'.
Saliendo del programa. ┬íHasta luego!

PS C:\Users\Israel\OneDrive\Desktop\pramacion en c++\examen2\output>
```